Génération automatique de playlist musicales par similarité

Rapport du projet semestriel
Encadré par Charles Brazier

Abdelhafid Guerouani
Jacques Xu
Maxime Gajic
Wassim Aarab

BORDEAUX INP Enseirb-Matmeca

Janvier 2026

Sommaire

1 Introduction **3**
1.1 Contexte et motivation 3
1.2 Problématique 3
1.3 Objectifs du projet 3

2 Etat de l'art **4**

3 Datasets **5**
3.1 Genre 5
3.2 Emotion 5
3.3 Multiclasse 5

4 Modélisation à partir d'un modèle non pré-entraîné **6**
4.1 Modèle de classification par Caractéristique 6
4.1.1 Stratégies d'entraînement 7
4.1.2 Classification en genre 7
4.1.3 Classification en émotion 7
4.1.4 Régression en émotion 7
4.2 Modèle de classification multiclasse from scratch 8

5 Modélisation supervisée à partir d'un modèle de fondation MuQ **9**
5.1 MuQ embeddings 9
5.2 Modèles de Classification par Caractéristique 9
5.2.1 Architecture des modèles de classification monoclasses . 10
5.2.2 Embeddings spécialisées 10
5.2.3 Stratégies d'entrainement 11
5.3 Modèle Globale Multiclasse 12
5.3.1 Architecture du modèle de classificaton multiclasse . . . 12
5.3.2 strategie d'entrainement 13

6 Experimentation et Analyse **14**
6.1 Performances par tâche 14
6.1.1 Classification en genre 14
6.1.2 Classification en émotion 15
6.1.3 Régression en émotion 17
6.2 Analyse de l'apport du multiclasse 18
6.2.1 Modèle from scratch 18
6.2.2 Modèle de fondation 20

7 Génération automatique des playlists **23**
7.1 Interprétation de l'espace musical 23
7.2 Construction du graphe musical 23
7.3 Algorithmes de génération de playlists 23

8 Méthodologie de mesure de l'empreinte carbone **24**
8.1 Protocole d'exécution 25
8.2 Métriques suivies 25

9 Axes d'améliorations **26**

10 Conclusion **26**

1 Introduction

1.1 Contexte et motivation

L'objectif de ce projet est d'implémenter un algorithme de machine learning capable de créer automatiquement des playlists musicales intelligentes à partir d'une base de données de morceaux variés. Les playlists générées proposeront une liste de musiques ordonnées par similarité.

Dans le cadre de notre projet sur la génération de playlists intelligentes, nous cherchons à dépasser les limites du filtrage collaboratif. Ce système traditionnel, basé sur l'historique des utilisateurs, délaisse les nouveautés et les titres peu connus : c'est le problème du "Cold Start". Pour y remédier, notre approche se concentre sur l'analyse intrinsèque du contenu musical. En extrayant directement les caractéristiques des morceaux, notre système permet de recommander des musiques récentes ou impopulaires, offrant ainsi une exploration plus personnalisée et diversifiée du catalogue.

1.2 Problématique

Comment l'analyse automatique des caractéristiques intrinsèques de l'audio par deeplearning peut résoudre le problème de "Cold start" afin de générer des playlist intelligentes basées sur le contenu?

1.3 Objectifs du projet

L'objectif principal de ce projet est d'apprendre un espace latent commun permettant de représenter les morceaux de musique de manière compacte et cohérente, de sorte que la distance entre deux représentations traduise leur degré de similarité musicale. Cette représentation vise à capturer des caractéristiques pertinentes liées au contenu audio telles que le genre, les instruments et l'émotion.

Dans cette optique, le projet s'attache à explorer et comparer différentes architectures et modèles de machine learning, afin d'évaluer leur capacité à produire des représentations discriminantes et interprétables. Une attention particulière est portée à l'analyse des couches intermédiaires des modèles, considérées comme porteuses d'informations sémantiques riches.

Les représentations apprises sont ensuite projetées dans un espace bidimensionnel à l'aide de la méthode de réduction de dimension t-SNE, dans le but de faciliter leur visualisation et construire une playlist musicale de proche en proche, en sélectionnant successivement des morceaux similaires, illustrant ainsi

l'intérêt pratique des représentations apprises pour des applications de recommandation musicale.

2 Etat de l'art

- **Haina Zhu, Yizhi Zhou, Hangting Chen, Jianwei Yu, Ziyang Ma, Rongzhi Gu, Yi Luo, Wei Tan, and Xie Chen. *MuQ: Self-supervised music representation learning with mel residual vector quantization.*** Cet article propose *MuQ*, un modèle de fondation pour l'apprentissage de représentations musicales par auto-supervision. Le modèle repose sur une quantification vectorielle résiduelle appliquée à des représentations Mel-spectrales, permettant d'apprendre des embeddings musicaux discrets et riches sans recourir à des annotations explicites. En capturant des structures musicales de haut niveau à partir de larges corpus audio non annotés, MuQ fournit des représentations génériques particulièrement adaptées à des tâches de Music Information Retrieval telles que la classification, la recherche par similarité ou la recommandation. Ce modèle constitue le socle méthodologique de notre travail, les embeddings qu'il génère servant de point de départ à une bonne partie des expérimentations menées dans ce projet.

- **Aaron Van den Oord, Sander Dieleman, and Benjamin Schrauwen. Deep content-based music recommendation :** Cet article explore des méthodes de recommandation musicale fondées sur le contenu audio des morceaux, en utilisant des réseaux de neurones pour prédire les facteurs latents. Nous nous sommes inspiré de ce papier pour la structure de notre modèle from scratch, en l'adaptant aux différentes taches et datasets.

- **A Short Survey and Comparison of CNN-Based Music Genre Classification Using Multiple Spectral Features :** Architecture Late Fusion combinant STFT, Mel-spectrogrammes et MFCC. Donne les résultats sur différents datasets

- **Limitations of interactive music recommendation based on audio content :** Cet article met en lumière le "fossé sémantique" (semantic gap) qui subsiste entre les caractéristiques acoustiques brutes et la perception humaine. Les auteurs soutiennent que, bien que l'analyse du signal audio (rythme, timbre, mélodie) soit efficace pour trouver des morceaux techniquement similaires, elle échoue fréquemment à capturer les dimensions contextuelles et émotionnelles essentielles à une recommandation interactive pertinente. L'étude souligne que les utilisateurs ne cherchent pas

seulement des sons "proches" mathématiquement, mais des expériences liées à une activité, une humeur ou une identité sociale, des éléments que les descripteurs audio seuls ne peuvent décoder. Par conséquent, l'article plaide pour une approche hybride, car s'appuyer uniquement sur le contenu audio limite la découverte à une "bulle de similarité acoustique" qui ignore la richesse de l'expérience culturelle de l'auditeur.

3 Datasets

Durant notre projet, nous avons utilisé différents datasets pour les différentes tâches effectuées et les différents modèles utilisés.

3.1 Genre

Nous avons utilisé le dataset **GTZAN** pour la classification en genre. Ce dataset contient 1000 extraits audios de 30 secondes, répartis en 10 genres de manière équilibrée.

3.2 Emotion

Nous avons utilisés différents datasets pour les tâches sur l'émotion, suivant le modèle et la tâche :

- Nous avons utilisé le dataset **DEAM** pour la régression. Ce dataset contient 1802 extraits audios avec un score de valence et d'arousal (entre 0 et 10) d'une durée d'environ 45 secondes
- Nous avons utilisé le dataset **4Q** pour la classification. Ce dataset est divisé en 4 émotions basées sur les Quadrants de Russell avec 225 extraits audios par émotions d'une durée d'environ 30 secondes.

3.3 Multiclasse

Les travaux présentés dans ce projet s'appuient sur le jeu de données MTG-Jamendo, un corpus de référence dédié à l'auto-annotation musicale, composé de plus de 55 000 morceaux audio issus de la plateforme Jamendo et distribués sous licences Creative Commons. Chaque piste est associée à des annotations sémantiques couvrant les catégories genre, instrument et ambiance/thème (mood/theme), pour un total de 195 étiquettes après les étapes de prétraitement décrites par les auteurs du dataset. Le dataset est accompagné de statistiques descriptives pré-calculées, fournies par les concepteurs du corpus. La Figure 1

illustre la répartition du nombre de pistes, d'albums et d'artistes par étiquette, triée selon le nombre d'artistes distincts associés à chaque tag, pour l'ensemble autotagging. Ces statistiques permettent de caractériser la structure du corpus et de mettre en évidence le déséquilibre de distribution des annotations, un aspect important qui impose la mise en place de stratégies de traitement spécifiques afin de pallier ce problème.

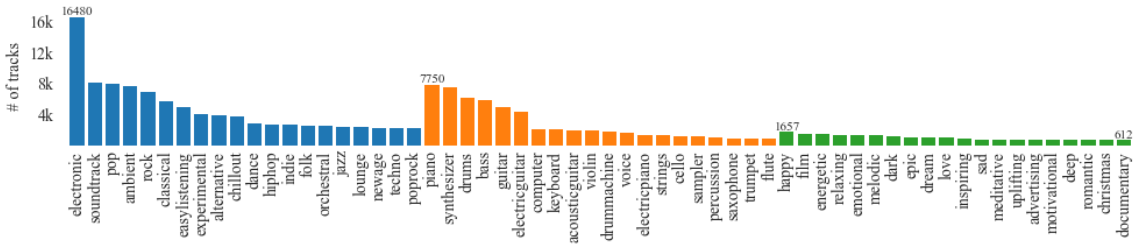


Figure 1: Statistiques du jeu de données MTG-Jamendo (ensemble *autotagging*) : Statistiques fournies par les auteurs du jeu de données, montrant la répartitions des tags.

4 Modélisation à partir d'un modèle non pré-entraîné

Nous nous sommes dans un premier temps intéressé à l'utilisation d'un modèle non pré-entraîné, que nous avons utilisé sur une classification en genre, et émotion, puis en multiclasse.

4.1 Modèle de classification par Caractéristique

Le modèle est inspiré du papier de recherche de Sander Dieleman développé dans la partie 2. Les couches restent les mêmes mais les sorties diffèrent en fonction des datasets. Les entrées dépendent de la durée des extraits audios sélectionnés, tandis que la sortie dépend du nombre de classe (on considère 2 classes pour la régression). Sur ce schéma, on peut voir les différentes couches du modèle utilisé, dans le cas de la classification en genre. Pour les autres caractéristiques, on aura la longueur du spectrogramme qui va changer (ici 1206) en fonction de la longueur des extraits audios, et la dimension de la couche de sortie (4 pour la classification en émotion, 2 pour la régression).

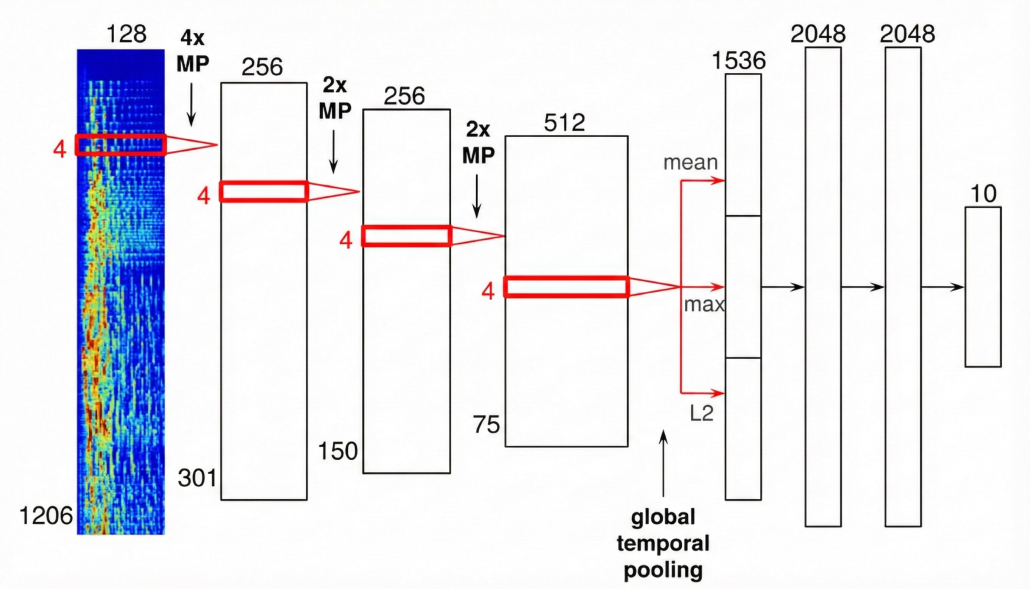


Figure 2: Schéma du CNN de classification en genre

4.1.1 Stratégies d'entraînement

Les datasets sont séparés en split train/validation/test, qui sont stratified afin de conserver l'equilibre des classes. L'entraînement se fait sur le dataset de train, avec des évaluations sur le dataset de validation. On effectue un early stopping si le modèle n'apprend plus, et on sauvegarde également le meilleur modèle sur le dataset de validation durant l'entraînement.

4.1.2 Classification en genre

La classification en genre se fait sur le dataset GTZAN avec des audios de 28 secondes. Le modèle prédit donc parmi 10 genres.

4.1.3 Classification en émotion

La classification en émotion se fait sur le dataset 4Q avec des audios de 30 secondes. Le modèle prédit donc parmi 4 classes.

4.1.4 Régression en émotion

Nous avons voulu essayer une régression en émotion, sur les scores de valence et d'arousal afin de comparer avec la simple classification. Cette régression se fait sur la dataset DEAM avec des audios de 45 secondes, et le modèle renvoie donc deux entiers. Nous avons également rajouté de la data augmentation afin d'améliorer les performances. Nous mesurons le score R2 pour évaluer les performances.

4.2 Modèle de classification multiclasse from scratch

Nous avons ensuite essayé de mettre en place un modèle de classification multiclasse, afin de voir l'effet que cela pouvait avoir sur les représentations t-SNE. Pour cela, nous avons choisi de combiner les tâches de classification de genre et de régression en émotion. Nous avons repris la même strucutre des modèles monoclasses, mais nous avons modifié la sortie pour avoir deux têtes : la première est celle du modèle de classification en genre, et la seconde est celle du modèle de régression en émotion. L'entrée est un audio d'une durée de 28 secondes ou 30 secondes selon la tâche. Cette différence de durée n'est pas problématique grâce au global pooling effectué avant les couches denses.

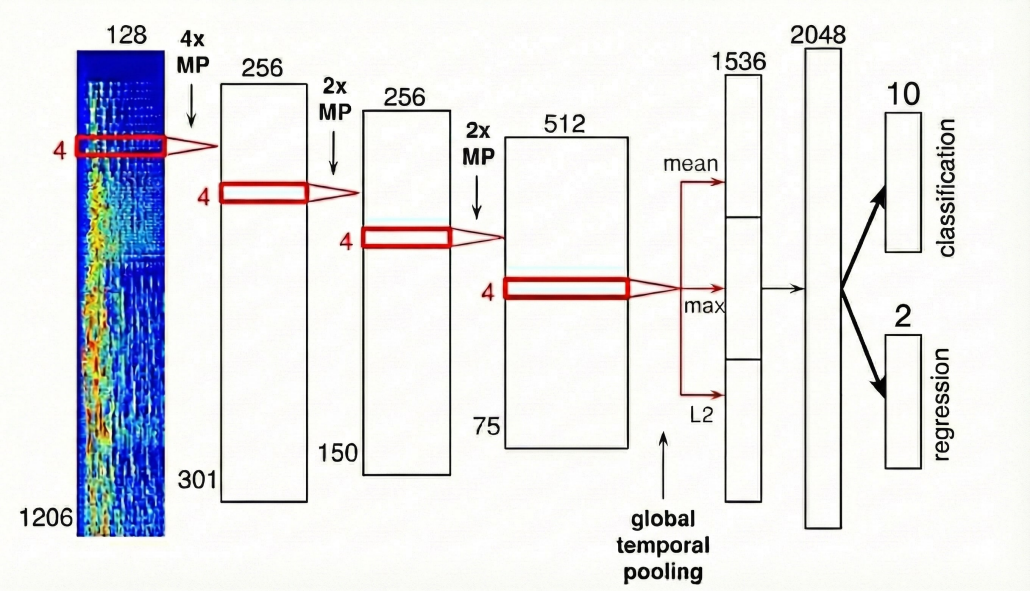


Figure 3: Schéma du CNN de classification multiclasse

L'entraînement s'effectue en alternant les batchs des deux datasets, qui sont également split en train/validation/test de manière stratified. Des loss différentes sont utilisées (CrossEntropy et MSE), et le meilleur modèle est choisi sur la somme des deux loss pour l'early stopping et la sauvegarde du modèle. Un poids est appliqué à la MSE pour équilibrer les deux loss, ainsi que des learning rate différents.

5 Modélisation supervisée à partir d'un modèle de fondation MuQ

5.1 MuQ embeddings

Les expériences menées dans ce projet s'appuient sur les embeddings MuQ, issus du modèle MuQ: Self-Supervised Music Representation Learning with Mel Residual Vector Quantization. MuQ est un modèle fondation pour la musique, pré-entraîné en auto-supervision (Self-Supervised Learning, SSL), dont l'objectif est d'apprendre des représentations musicales riches et génériques à partir du signal audio brut. Le modèle repose sur une stratégie originale de quantification, Mel Residual Vector Quantization (Mel-RVQ), utilisée comme cible d'apprentissage auto-supervisé, ce qui lui permet d'atteindre des performances de pointe sur de nombreuses tâches de Music Information Retrieval (MIR).

Les embedding MuQ correspondent aux représentation latente produite par le modèle à partir d'un signal audio musical échantillonné à 24 kHz. Grâce à son pré-entraînement à grande échelle, MuQ fournit des embeddings transférables, exploitables pour diverses tâches en aval, notamment la classification, l'annotation automatique ou la recommandation musicale.

Pour un extrait audio de 30 secondes, MuQ génère un embedding global représentant le contenu musical du signal. Cette représentation, extraite de la sortie principale (last hidden state), est un vecteur de dimension 1024, correspondant à la dimension de l'espace latent du modèle.

Nous avons mis en place un pipeline standard d'extraction des embeddings pour l'ensemble des jeux de données. Le signal audio est d'abord chargé puis ré-échantillonné à 24 kHz, conformément aux contraintes du modèle. Il est ensuite transmis au modèle MuQ pré-entraîné, utilisé en mode inférence. Les sorties des couches internes peuvent être récupérées, permettant d'exploiter la représentation finale. Dans ce projet, les embeddings sont extraits sans ajustement des poids, MuQ étant utilisé comme extracteur de caractéristiques figé.

Les vecteurs d'embeddings obtenus sont ensuite enregistrés dans un fichier CSV, auquel sont ajoutées les étiquettes nécessaires. Ce fichier constitue la base pour l'entraînement des modèles utilisés dans ce projet

5.2 Modèles de Classification par Caractéristique

Afin d'exploiter les vecteurs d'embeddings extraits et d'évaluer leur valeur ajoutée pour la réalisation de tâches de Music Information Retrieval (MIR), nous avons mis en place plusieurs modèles de classification, chacun dédié à la prédiction d'une caractéristique musicale spécifique, à savoir le genre, l'émotion et les instruments.

5.2.1 Architecture des modèles de classification monoclasses

Pour chacune des tâches de classification considérées (genre, émotion et instrument), nous utilisons la même architecture de classification, appliquée indépendamment à chaque jeu d'étiquettes. Cette approche permet d'évaluer de manière cohérente la capacité des embeddings MuQ à capturer différentes caractéristiques musicales, tout en conservant un cadre expérimental homogène.

Chaque modèle de classification prend en entrée un embedding MuQ de dimension 1024, extrait préalablement et utilisé comme représentation fixe du signal audio. Ces embeddings sont fournis à un perceptron multicouche (MLP) constitué de trois couches entièrement connectées. Afin d'améliorer la stabilité de l'apprentissage et la généralisation du modèle, chaque couche cachée est suivie d'une normalisation par lot (Batch Normalization), d'une fonction d'activation ReLU, ainsi que d'un mécanisme de Dropout avec un taux fixé à 0.3. Ce choix architectural vise à limiter le sur-apprentissage tout en conservant une capacité de représentation suffisante.

5.2.2 Embeddings spécialisées

Au-delà de la prédiction finale, nous exploitons également la sortie de la dernière couche cachée du classifieur, de dimension 256, comme un nouvel embedding appris. Ce vecteur peut être interprété comme une représentation latente spécialisée, optimisée pour la caractéristique musicale considérée (genre, émotion ou instrument). Contrairement aux embeddings MuQ génériques, cette représentation est affinée par apprentissage supervisé, et capture donc des informations plus discriminantes vis-à-vis de la tâche ciblée.

Ainsi, cette approche permet non seulement d'évaluer les performances de classification, mais également de projeter les morceaux dans un nouvel espace latent, plus pertinent pour la caractéristique étudiée. Ces embeddings spécialisés peuvent ensuite être exploités pour des tâches d'analyse et de visualisation, notamment à l'aide de la méthode t-SNE, offrant une comparaison particulièrement pertinente avec les embeddings d'entrée. Voir figure

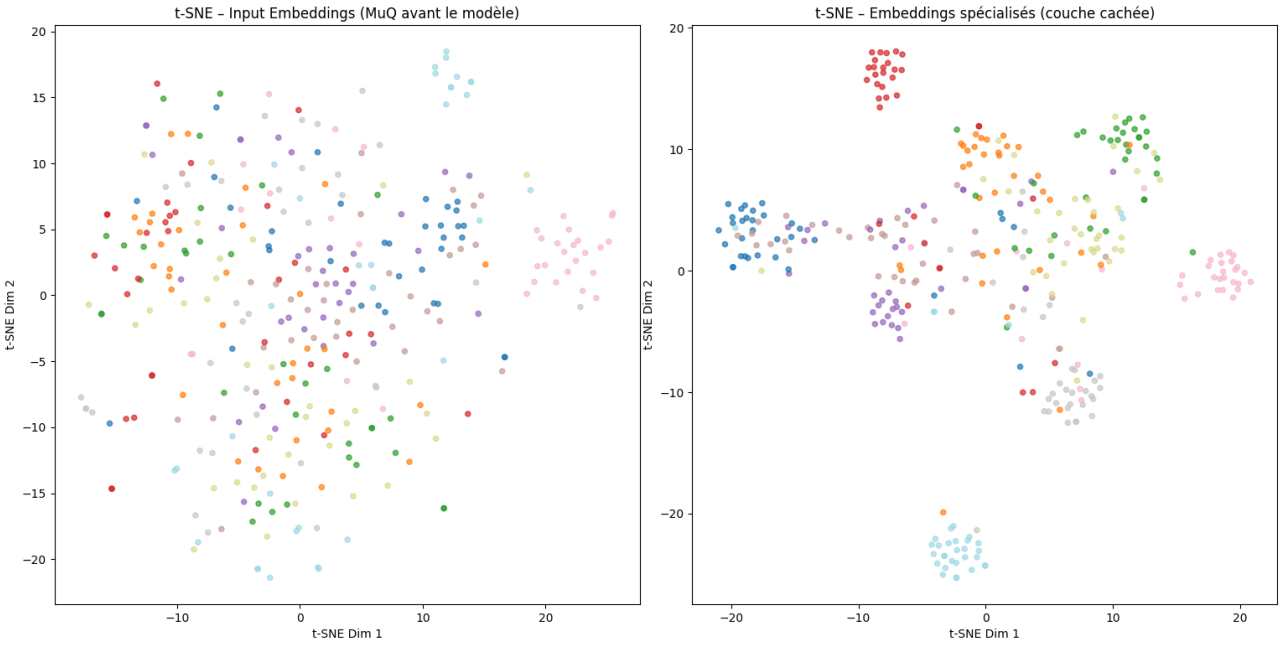


Figure 4: Visualisation par t-SNE des embeddings MuQ et des embeddings spécialisés issus du modèle entraîné pour la classification d'instruments sur le jeu de données Jamendo.

5.2.3 Stratégies d'entrainement

Les embedding préalablements extraits et stockés avec leurs labels dans fichier csv constituent notre jeux de donnée. Ce jeux de données est divisée en trois en trois sous-ensembles disjoints : 70% pour l'entrainement, 15% pour la validation et 15% pour le test. Cette séparation est réalisée de manière stratifée, garantissant une distribution homogène des classes de chaque sous-ensemble.

L'entraînement est effectué par mini-lots, avec un mélange aléatoire des données uniquement pour l'ensemble d'entraînement, afin de favoriser une meilleure généralisation du modèle.L'apprentissage repose sur la fonction de perte Cross-Entropy, adaptée aux problèmes de classification multiclasse. L'optimisation est réalisée à l'aide de l'algorithme Adam, avec un taux d'apprentissage initial fixé à 0.001 et une régularisation de type weight decay afin de limiter le sur-apprentissage.

Un scheduler ReduceLROnPlateau est utilisé pour ajuster automatiquement le taux d'apprentissage en fonction des performances sur l'ensemble de validation. Cette stratégie permet de stabiliser l'entraînement et d'améliorer la convergence du modèle lorsque les performances stagnent.

À chaque époque, les performances sont évaluées sur l'ensemble de validation. Le modèle obtenant la meilleure précision de validation est sauvegardé, ce qui garantit que la version finale du classifieur correspond au meilleur com-

promis entre capacité de généralisation et performance.

5.3 Modèle Globale Multiclasse

L'objectif principal du modèle global multiclasse est d'apprendre des embeddings de similarité riches et informatifs, capables de capturer simultanément plusieurs dimensions musicales du signal audio. En entraînant un encodeur partagé sur des tâches complémentaires telles que la prédiction du genre, des instruments et des émotions, le modèle est encouragé à construire une représentation latente intégrant à la fois des informations timbrales, structurelles et expressives. Ces embeddings multi-tâches offrent ainsi une mesure de similarité plus pertinente entre morceaux, dans la mesure où deux signaux proches dans l'espace latent partagent non seulement des caractéristiques acoustiques, mais également des propriétés musicales de plus haut niveau. Cette représentation commune constituera le support de base pour la génération de playlist.

5.3.1 Architecture du modèle de classificaton multiclasse

Le modèle global multiclasse repose sur une architecture *multi-tâches* visant à apprendre simultanément plusieurs caractéristiques musicales à partir d'une représentation audio commune. Cette approche s'appuie sur le partage d'un encodeur profond suivi de têtes de classification spécialisées, une stratégie largement adoptée afin d'exploiter les corrélations existantes entre différentes tâches tout en préservant une spécialisation des sorties pour chaque objectif de classification (voir Figure 5).

L'encodeur constitue le cœur du modèle et est partagé entre l'ensemble des tâches considérées, à savoir la classification du genre, des instruments et des émotions. Il prend en entrée un vecteur d'embedding audio de dimension fixe, par exemple de dimension 1024 dans le cas des embeddings MuQ, et le projette dans un espace latent de dimension réduite. Cet encodeur est implémenté sous la forme d'un perceptron multicouche (MLP) composé de trois couches entièrement connectées de dimensions successives $[512, 256, 128]$.

Afin d'améliorer la stabilité de l'apprentissage et la capacité de généralisation du modèle, chaque couche cachée est suivie d'une normalisation par lot (*Batch Normalization*), d'une fonction d'activation non linéaire de type ReLU, ainsi que d'un mécanisme de régularisation par *Dropout*. La sortie de la dernière couche cachée définit un embedding latent partagé, qui constitue une représentation compacte et expressive du signal audio, optimisée conjointement pour l'ensemble des tâches.

À partir de cet embedding partagé, le modèle intègre plusieurs têtes de classification indépendantes, chacune associée à une caractéristique musicale

spécifique. Chaque tête est implémentée par une couche linéaire projetant l'espace latent commun vers l'espace des classes correspondant à la tâche considérée.

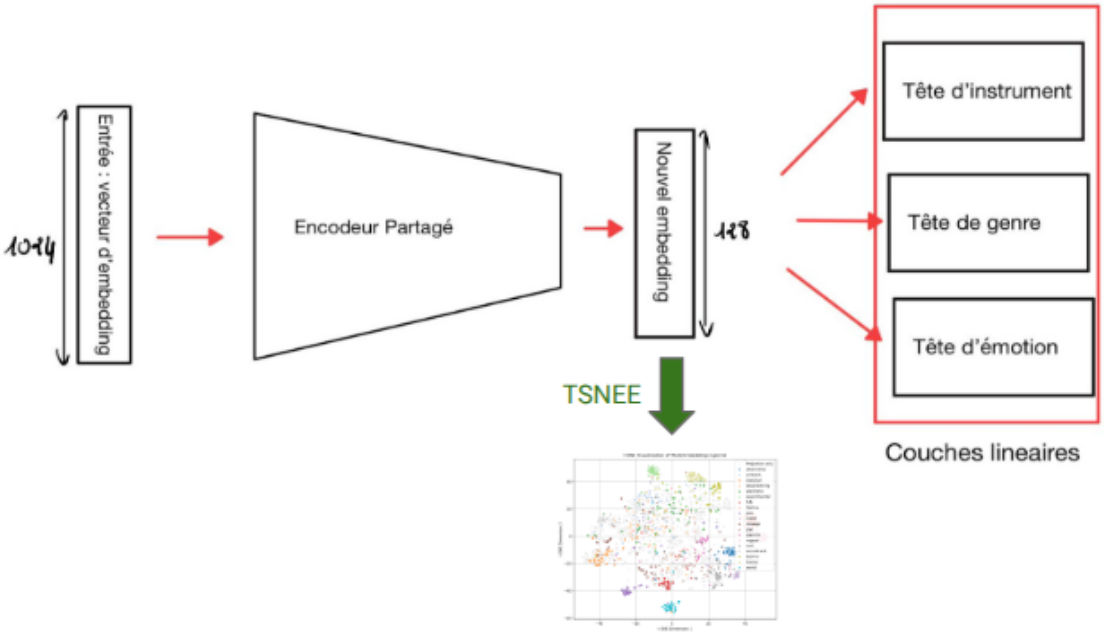


Figure 5: Architecture du modèle global multiclasse basé sur un encodeur partagé et des têtes de classification spécialisées.

5.3.2 strategie d'entrainement

L'entraînement du modèle repose sur une approche *multi-tâches*, où plusieurs objectifs de classification peuvent être appris simultanément. À chaque époque, le modèle est entraîné sur des mini-lots d'embeddings audio, et seules les tâches explicitement activées participent au calcul de la perte. Pour chaque tâche active, une fonction de perte de type *cross-entropy* est calculée indépendamment à partir des sorties correspondantes du modèle. La perte totale est ensuite obtenue par une combinaison linéaire pondérée de ces pertes, permettant de contrôler l'influence relative de chaque tâche lors de l'optimisation. Cette perte globale est utilisée pour la rétropropagation du gradient et la mise à jour des paramètres du modèle via un optimiseur. En parallèle, les performances sont évaluées en calculant l'exactitude de classification pour chaque tâche active, ce qui permet de suivre séparément l'évolution de l'apprentissage pour chaque objectif.

6 Experimentation et Analyse

Nous allons désormais nous intéresser aux performances des modèles dans les différents tâches.

6.1 Performances par tâche

Les performances sont réparties par caractéristiques, puis par modèle.

6.1.1 Classification en genre

Pour mesurer les performances des modèles en genre, nous avons utilisé l'accuracy du modèle sur le dataset de test.

Modèle from scratch : Voici les courbes de loss et d'accuracy durant l'entraînement du modèle.

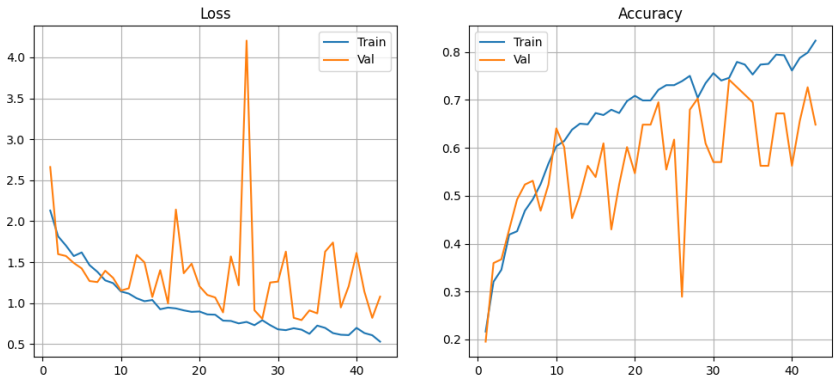


Figure 6: Courbes d'entraînement du modèle from scratch de classification en genre

Le modèle se rapproche des 75% d'accuracy sur le dataset de test, ce qui est une performance correcte. Cela nous donne une projection t-SNE comme cela.

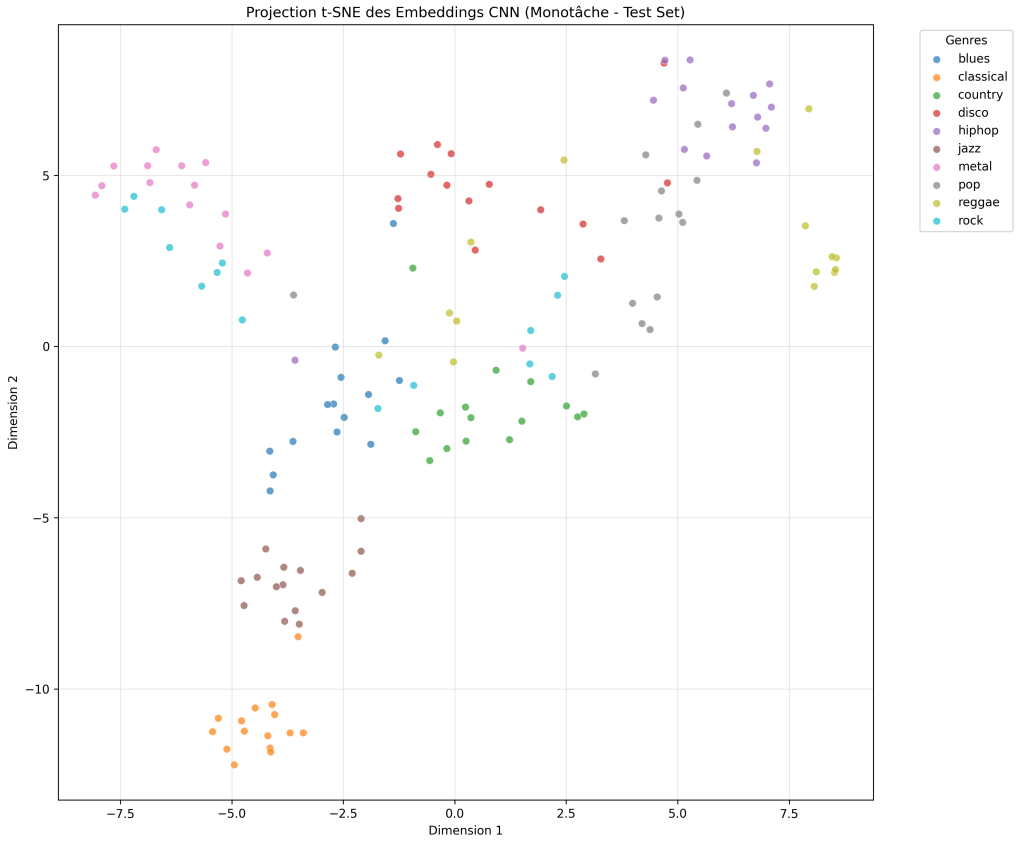


Figure 7: Projection t-SNE du modèle from scratch de classification en genre

On observe que certains genres sont bien regroupés (classical), mais on observe pas de clusters distincts pour certains genres.

6.1.2 Classification en émotion

Pour mesurer les performances des modèles, nous avons utilisé l'accuracy du modèle sur le dataset de test.

Modèle from scratch : Voici les courbes de loss et d'accuracy durant l'entraînement du modèle.

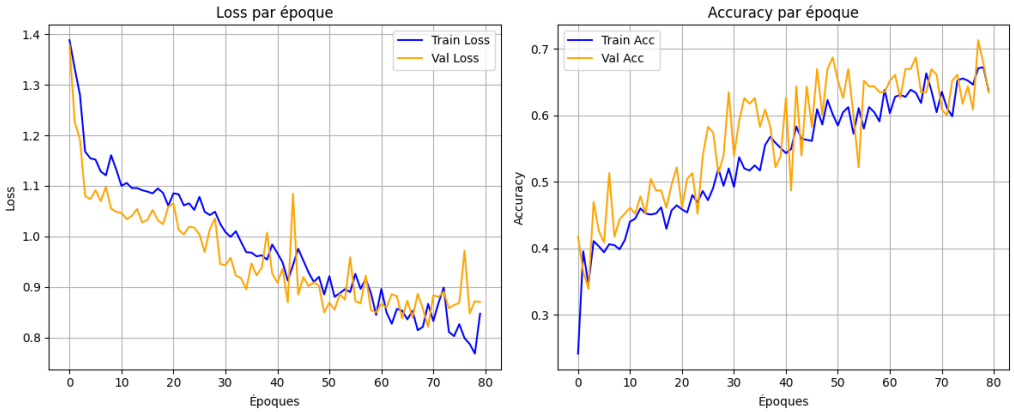


Figure 8: Courbes d'entraînement du modèle from scratch de classification en émotion

On observe des performances atteintes de 65% d'accuracy sur le dataset de test. On observe que le modèle prédit bien les classes aux extrèmes, contrairmeent aux classes intermédiaires. Ces performances, inférieures à celles sur le genre (moins de classes), montrent que la classification en émotion est plus difficiles que celle en genre en se basant sur les spectrogrammes. Voici les projections t-SNE sur le dataste de test.

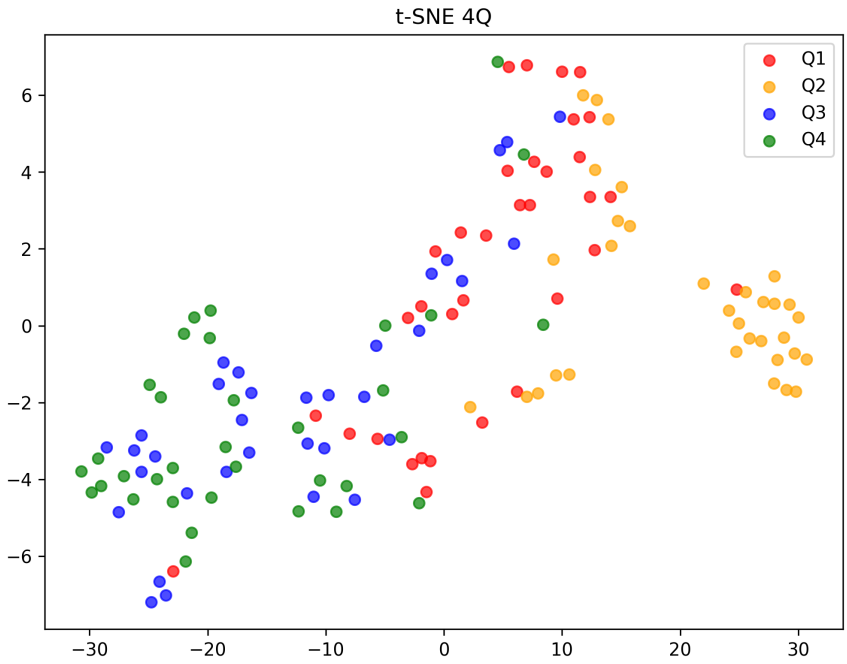


Figure 9: Projection t-SNE du modèle from scratch de classification en émotion

On observe une séparation correcte des classes, même si certaines se mélangent un peu.

6.1.3 Régression en émotion

Pour le modèle from scratch, une tâche de régression en émotion a été testée. Voici les courbes de performances pendant l'entraînement du modèle. Nous avons mesuré le coefficient R2 sur le dataset de test.

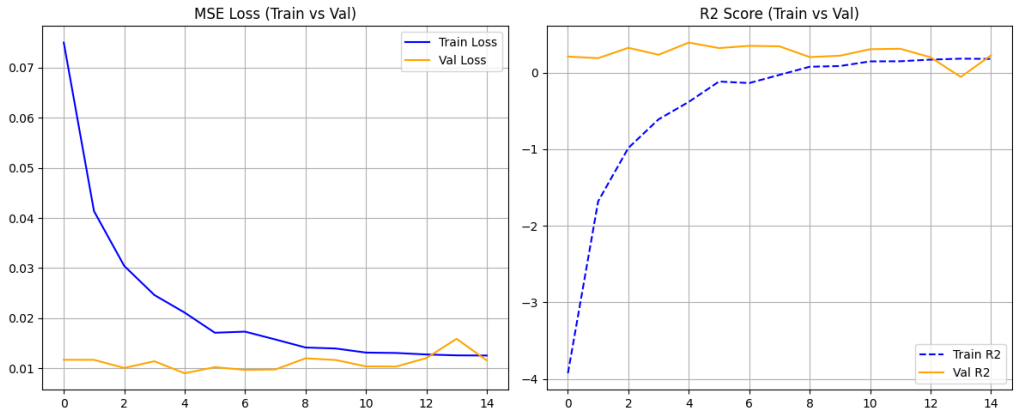


Figure 10: Courbes d'entraînement du modèle from scratch de régression en émotion

On observe un R2 moyen de d'environ 0.4 sur le dataset de test, avec des performances meilleures pour l'arousal. Voici les régressions selon les deux métriques

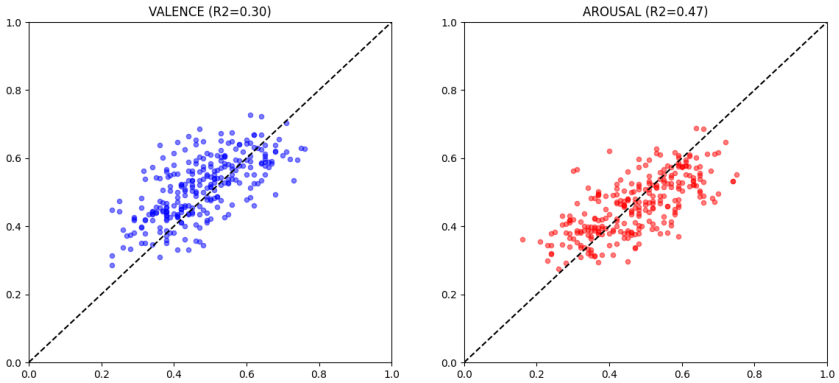


Figure 11: Régression des scores d'arousal et de valence

Concernant la visualisation t-SNE, voici les projections en fonction de chacun des scores, puis des quadrants de Russel.

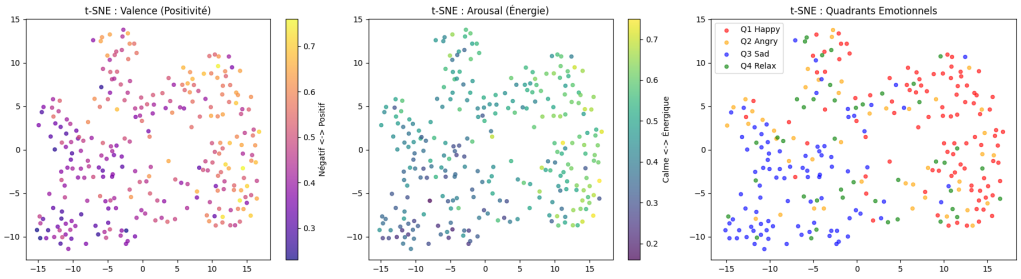


Figure 12: Projection t-SNE du modèle from scratch de régression en émotion

On observe que les classes extrêmes sont bien distinguées, contrairement aux classes intermédiaires. Pour comparer à la classification en émotion, on observe une meilleure séparation des classes, c'est pourquoi c'est la tâche que nous avons choisi pour le modèle multiclasse.

6.2 Analyse de l'apport du multiclasse

Nous allons désormais analyser les performances du modèle multiclasse.

6.2.1 Modèle from scratch

Voici les courbes d'entraînements du modèle. On observe à gauche la loss et à droite la métrique (accuracy et R2).

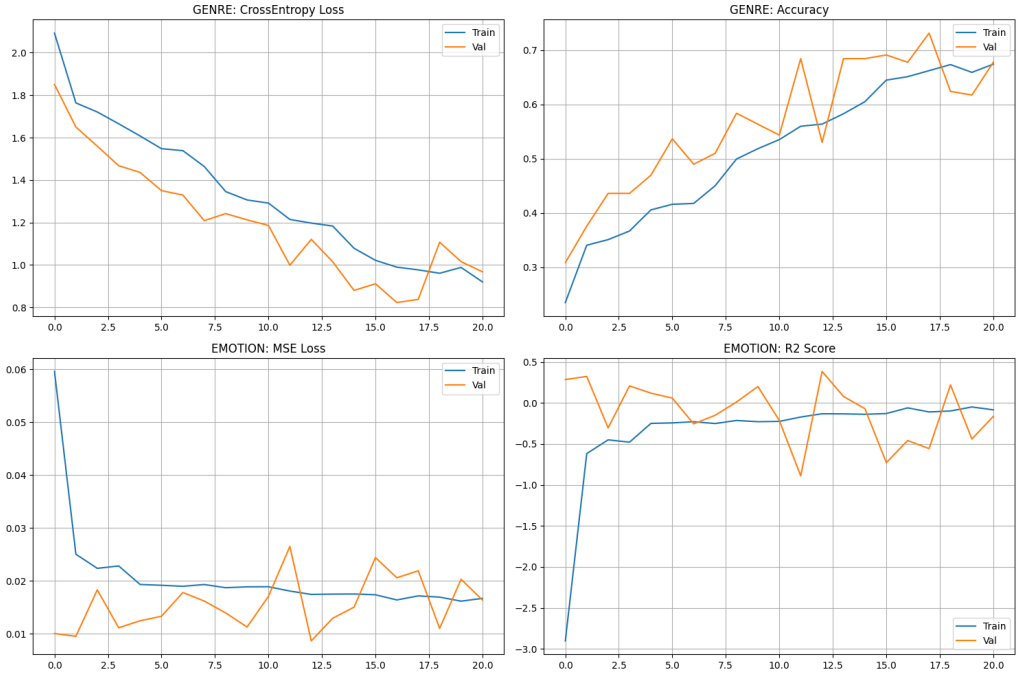


Figure 13: Courbes d'entraînement du modèle from scratch multiclasse

On observe que les performances du modèles progressent pendant l'entrainement dans les deux tâches. Sur le dataset de test, on s'approche des 70% d'accuracy en genre et 0.35 de score R2. On observe une diminution du score R2 et de l'accuracy. Cela peut être interprété par le fait que se diversifier dans les tâches fait perdre de la specialisation au modèle. Voici la projection t-SNE faite sur la concaténation des deux datasets de test.

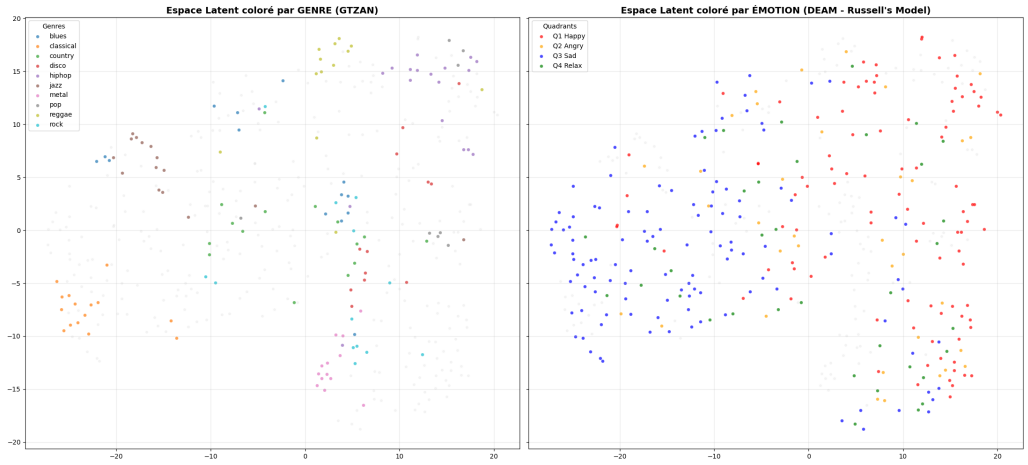


Figure 14: Projection t-SNE du modèle from scratch multiclasse

On observe des clusters en genre et en émotion, avec des genres qui sont situés au niveau de l'émotion qui leur est associée. Pour tester l'impact du multiclasse, nous avons voulu évaluer les performances et visualisations t-SNE lors d'un entraînement uniquement sur l'une des deux tâches, puis un test sur les deux autres. Cela nous donne deux nouvelles visualisations.

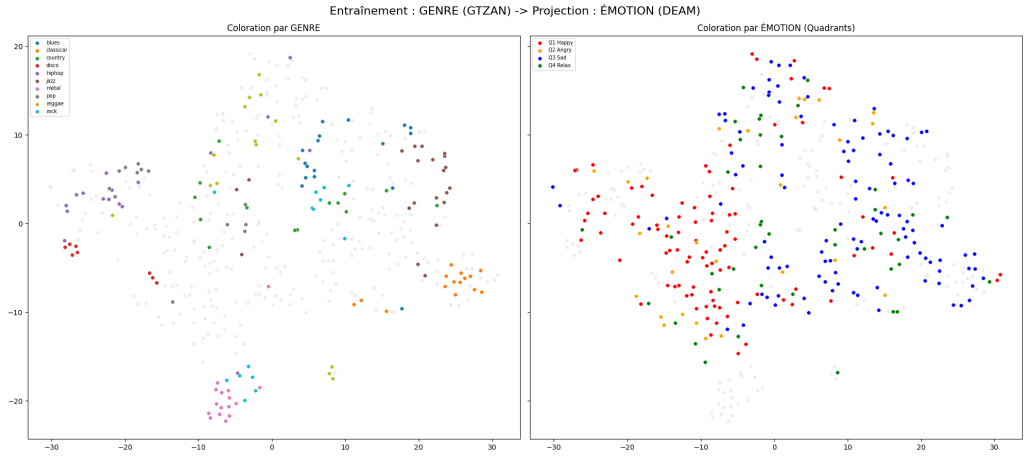


Figure 15: Projection t-SNE du modèle from scratch multiclasse sur le genre

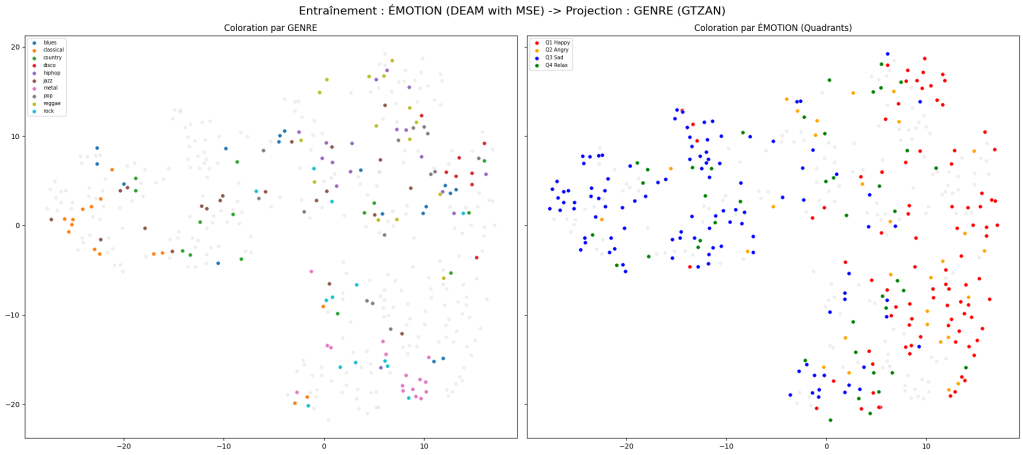


Figure 16: Projection t-SNE du modèle from scratch multiclasse sur l'émotion

Lors de l'entraînement en genre, on a un R2 de -21 sur l'émotion, et pour l'autre configuration on a une accuracy de 2% sur le genre. Les résultats sont donc inférieurs au hasard. Concernant les projections, on observe malgré tout que lors des entraînements sur une seule tâche, le modèle arrive quand même à une légère séparation pour l'autre tâche. Cela montre que les tâches ont des caractéristiques spectrales similaires que le modèle arrive à distinguer peu importe la tâche, même sans savoir classer les audios.

6.2.2 Modèle de fondation

Le modèle multi-tâches a été entraîné sur trois tâches de classification distinctes avec des performances variables selon la complexité de chaque tâche. Pour la classification de **genres musicaux**, le modèle atteint une précision de *69.3%* sur l'ensemble de test, avec une perte finale de *0.0527*. Concernant la classification des **instruments**, les résultats montrent une précision de *68.3%* en test, avec une perte de *0.0609*. Enfin, pour la détection d'**émotions**, le modèle obtient une précision de *77.6%* en test, avec une perte de *0.0522*. L'écart entre les performances d'entraînement et de test reste limité pour toutes les tâches, suggérant une bonne capacité de généralisation du modèle. La Figure 17 illustre l'évolution des pertes et des précisions au cours de l'entraînement.

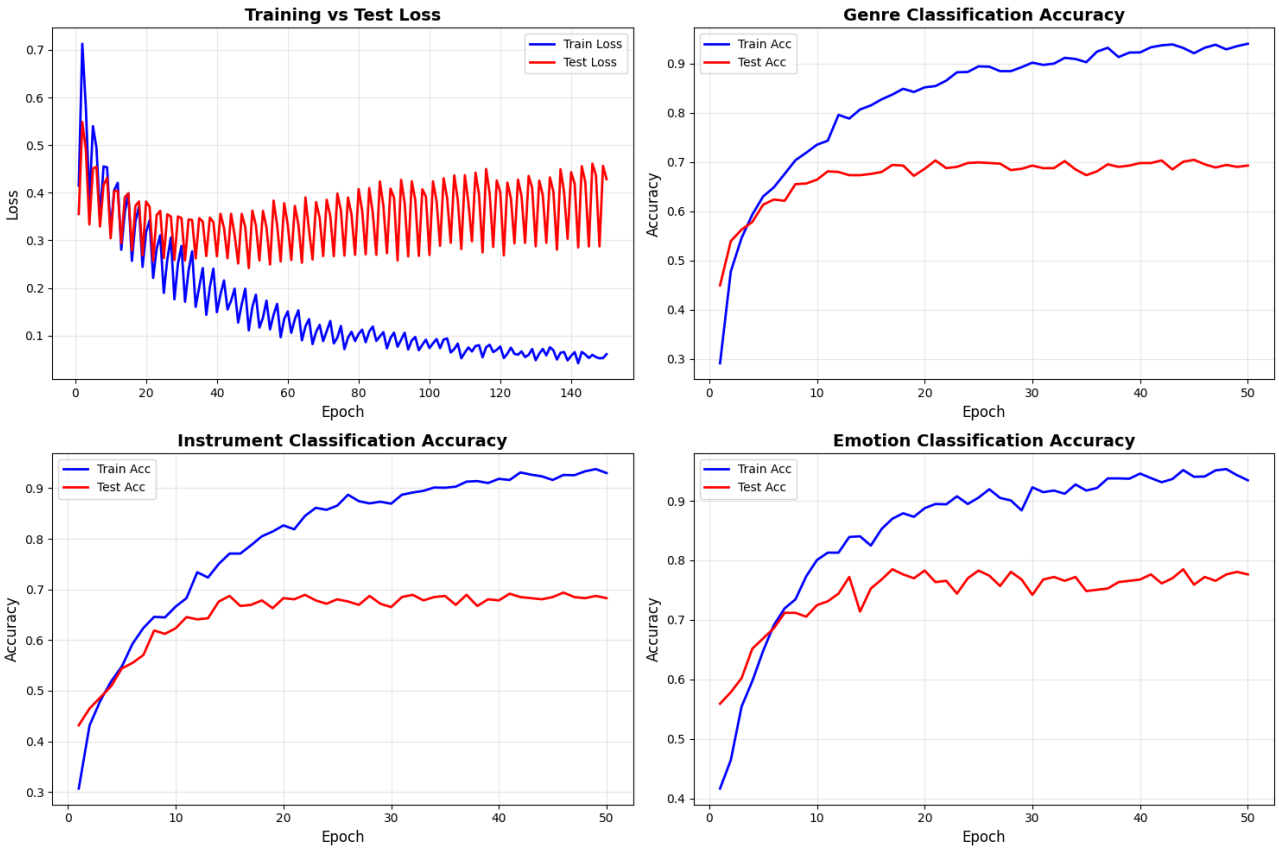


Figure 17: Évolution des pertes et des précisions pour les trois tâches de classification au cours de l'entraînement.

La Figure 18 présente la projection t-SNE des embeddings latents partagés du jeu de données de test, appris par le modèle multi-tâches. Chaque sous-figure correspond à une coloration des points selon les étiquettes associées à une tâche spécifique (genre, instruments ou émotions).

On observe que les embeddings appris tendent à former des regroupements cohérents dans l'espace latent, en particulier pour certaines classes bien définies. Cette structuration indique que l'encodeur partagé parvient à capturer des régularités musicales discriminantes communes aux différentes tâches. La séparation partielle des classes, malgré l'apprentissage conjoint de plusieurs objectifs, suggère que le modèle apprend une représentation de similarité riche, intégrant simultanément des informations liées au timbre, à l'instrumentation et au contenu expressif des morceaux.

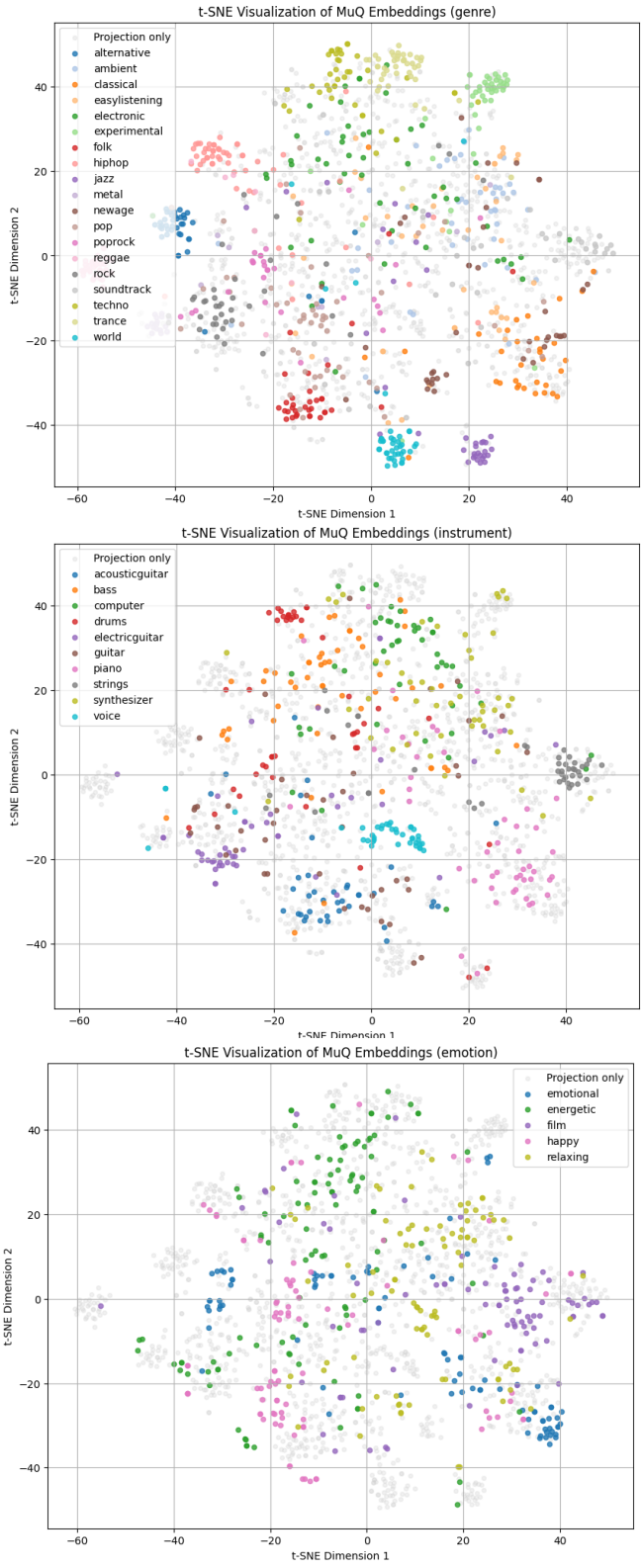


Figure 18: Projections t-SNE des embeddings latents partagés, colorées respectivement selon les labels de genre, d'instruments et d'émotions.

7 Génération automatique des playlists

7.1 Interprétation de l'espace musical

L'espace latent obtenu à partir des embeddings MuQ spécialisés permet de représenter chaque morceau par un vecteur $\mathbf{e}_i \in R^D$. La proximité entre deux morceaux dans cet espace, mesurée par la similarité cosinus entre leurs embeddings, reflète la similarité musicale perçue : des morceaux proches présentent des caractéristiques communes en termes de genre, émotion ou instrumentation.

Pour faciliter l'interprétation et la visualisation, nous projetons ces embeddings dans un espace bidimensionnel à l'aide de *t-SNE*, noté $\mathbf{z}_i \in R^2$. Dans cet espace réduit, les relations de proximité entre morceaux restent globalement préservées, permettant de visualiser des clusters de morceaux similaires et d'assurer des transitions cohérentes lors de la génération automatique de playlists.

- proximité musicale perçue : reflétée par la distance entre les embeddings dans l'espace latent. Elle sera exploitée pour assurer des transitions cohérentes lors de la génération automatique des playlists.

7.2 Construction du graphe musical

À partir de la projection t-SNE de cet espace musicale, nous construisons un graphe musical $G = (\mathcal{V}, \mathcal{E})$ où chaque nœud $v_i \in \mathcal{V}$ correspond à un morceau m_i. Les arêtes $(v_i, v_j) \in \mathcal{E}$ relient les morceaux proches et sont pondérées par une mesure de similarité, par la distance euclidienne :

$$w_{ij} = d(\mathbf{z}_i, \mathbf{z}_j),$$

où $d(\cdot, \cdot)$ est la distance euclidienne.

Pour capturer les relations locales tout en limitant la densité du graphe, un graphe des k plus proches voisins (k-NN) est construit :

$$(v_i, v_j) \in \mathcal{E} \iff \mathbf{z}_j \in \mathcal{N}_k(\mathbf{z}_i),$$

où $\mathcal{N}_k(\mathbf{z}_i)$ désigne l'ensemble des k points les plus proches de $\mathbf{z}_i$ dans l'espace t-SNE. Cette construction permet de préserver les similarités musicales pertinentes tout en offrant une représentation visuelle interprétable du corpus.

7.3 Algorithmes de génération de playlists

Les playlists sont générées en naviguant de proche en proche dans l'espace de représentation obtenu par projection t-SNE des embeddings musicaux. Cette

approche consiste à sélectionner, à partir d'un morceau de départ, des titres successifs présentant une forte similarité dans l'espace latent, tout en tenant compte d'un paramètre de diversité.

La Figure 19 illustre un exemple de cinq playlists générées à partir du même point d'ancrage, en faisant varier le coefficient de diversité dans l'espace latent. Ce coefficient permet de moduler le compromis entre continuité musicale et exploration de nouveaux morceaux.

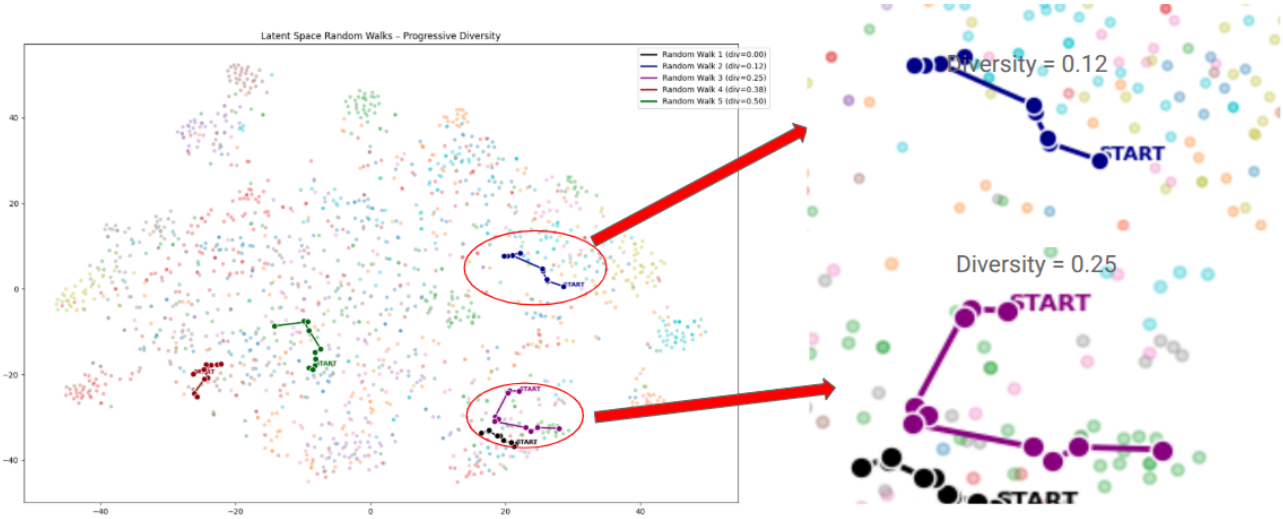


Figure 19: Exemple de playlists générées par navigation dans l'espace latent projeté par t-SNE.

On observe clairement le parcours suivi au sein de la projection t-SNE pour construire chaque playlist, traduisant une progression cohérente basée sur la similarité musicale. L'influence du coefficient de diversité est également visible : des valeurs faibles favorisent des transitions locales entre morceaux proches, tandis que des valeurs plus élevées permettent d'explorer des régions plus éloignées de l'espace latent, introduisant ainsi une plus grande variété musicale au sein des playlists générées.

8 Méthodologie de mesure de l'empreinte carbone

Afin d'évaluer l'impact environnemental de nos entraînements de modèles, nous avons intégré l'outil **CodeCarbon**. Cet outil permet d'estimer la quantité de CO_2 émise en fonction de la consommation électrique des composants matériels (CPU, GPU et RAM) et de l'intensité carbone du mix énergétique local.

8.1 Protocole d'exécution

Pour automatiser la collecte des métriques sans modifier le code source des modèles, nous utilisons un script wrapper nommé `run_with_emissions.py`. Ce script encapsule l'exécution de l'entraînement et génère un fichier `emissions.csv` contenant les données brutes.

La commande d'exécution utilisée est la suivante :

```
python run_with_emissions.py your_training_file.py --project name "your p
```

8.2 Métriques suivies

Les indicateurs clés extraits pour chaque expérience sont :

- **Émissions ($kgCO_2eq$)** : L'équivalent carbone total généré.
- **Energy Consumed (kWh)** : L'énergie électrique totale consommée par le serveur.
- **Duration (s)** : Le temps total d'exécution de l'entraînement.

Ces données sont ensuite traitées et visualisées via un tableau de bord *Streamlit* pour comparer l'efficacité énergétique de différentes architectures de systèmes experts ou de modèles de réseaux de neurones.

La commande pour lancer l'application est la suivante à la racine du dépot:

```
streamlit run app.py
```



Figure 20: Résultats globaux des entrainements

	timestamp	project_name	duration	emissions
0	2025-12-10 22:03:54	CNN_4Q_training	169.643924	0.0009
1	2025-12-10 22:07:36	CNN_4Q_training	149.226827	0.0008
2	2025-12-10 22:17:49	CNN_4Q_training	237.254071	0.0012
3	2025-12-11 15:32:59	CNN_4Q_training	23.633906	0.0001
4	2025-12-11 15:35:45	CNN_4Q_training	15.528069	0.0001
5	2025-12-11 16:23:15	CNN_4Q_training	15.480843	0.0001
6	2025-12-11 16:38:53	CNN4Q_training	20.564012	0.0001
7	2025-12-11 16:49:12	CNN_multi_training	19.664953	0.0001
8	2025-12-11 16:54:24	CNN_multi_training	40.241335	0.0002
9	2025-12-11 16:55:48	CNN_multi_training	41.836329	0.0002

Figure 21: Tableau des consommations pour chaque entrainement

Ce tableau montre les dates de chaque entrainement, la durée, l'emission de C02 et la consommation énergétique. On constate que l'impact reste minime pour un entrainement mais en cumulé on atteint des quantités plus importantes d'emission de C02. Concernant la consommation du modèle MUQ, ce n'est pas l'entrainement qui coute le plus mais le préentrainement du modèle. D'après les informations trouvées sur internet, l'entrainement de MUQ aurait requis entre 8 et 32 GPU NVIDIA V100 pour une durée d'entrainement entre 5 et 15 jours consommant 3000 kWH ce qui correspond à la consommation annuelle moyenne d'un français soit 1 tonne C02.

9 Axes d'améliorations

Un axe d'amélioration possible pourrait être d'augmenter le nombre de caractéristiques apprises du modèle multiclasse. Pour l'instant, nos modèles regroupent deux caractéristiques, mais cela pourrait être intéressant de tester avec 3 ou plus.
Un deuxième axe d'amélioration possible pourrait être la méthode de génération de playlist. En effet, nous sommes resté sur un simple parcours de graphe par proche voisin, mais nous pourrions prendre également l'historique d'un utilisateur en compte, pour améliorer la pertinence des playlists.

10 Conclusion

Ce projet a permis de générer automatiquement des playlists musicales basées sur la similarité entre morceaux, en exploitant des embeddings audio appris par MuQ et des architectures multiclasse. Les représentations latentes obtenues capturent efficacement le genre, l'émotion et l'instrumentation, permettant des transitions cohérentes et une diversité contrôlée dans les playlists.

L'approche démontre que l'analyse du contenu audio peut dépasser les limites du filtrage collaboratif et résoudre le problème de "Cold Start", tout en offrant des recommandations musicales interprétables et personnalisables. Les travaux futurs pourraient intégrer davantage de dimensions musicales et combiner préférences utilisateurs et contenu pour enrichir encore la recommandation.